

5. Cycles dans les graphes

Voici un rappel de deux définitions :

- Une chaîne est une suite d'arêtes consécutives dans un graphe, un peu comme si on se promenait sur le graphe. On la désigne par les lettres des sommets qu'elle comporte. On utilise le terme de chaîne pour les graphes non orientés et le terme de chemin pour les graphes orientés.
- Un cycle est une chaîne qui commence et se termine au même sommet.

Pour différentes raisons, il peut être intéressant de détecter la présence d'un ou plusieurs cycles dans un graphe (par exemple, pour savoir s'il est possible d'effectuer un parcours qui revient à son point de départ sans être obligé de faire demi-tour).

Nous allons étudier un algorithme qui permet de "détecter" la présence d'au moins un cycle dans un graphe :

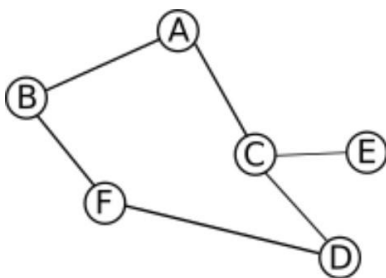
```
VARIABLES
s : nœud (nœud quelconque)
G : un graphe
u : nœud
v : nœud
p : pile (vide au départ)
//On part du principe que pour tout sommet u du graphe G, u.couleur = blanc à l'origine

DEBUT
CYCLE() :
  empiler(s, p)
  tant que p n'est pas vide :
    u ← depiler(p)
    pour chaque sommet v adjacent au sommet u :
      si v.couleur n'est pas noir :
        empiler(v, p)
      fin si
    fin pour
    si u est noir:
      renvoie Vrai
    sinon:
      u.couleur ← noir
    fin si
  fin tant que
  renvoie Faux
FIN
```

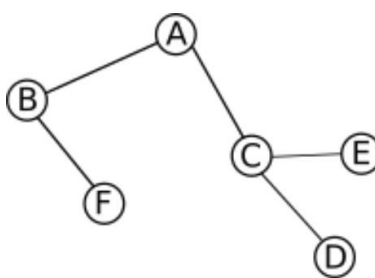
1) Étudier cet algorithme :

- Que se passe-t-il quand le graphe G contient au moins un cycle ?
- Que se passe-t-il quand le graphe G ne contient aucun cycle ?

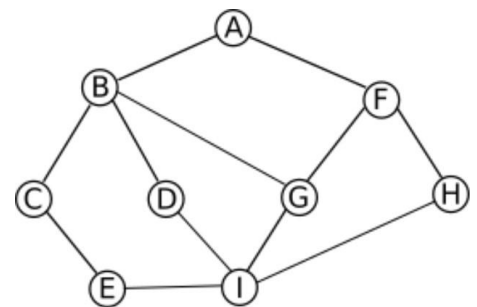
2) Appliquer l'algorithme de détection d'un cycle à chacun des trois graphes ci-dessous, en partant du sommet de votre choix.



graphe 1



graphe 2



graphe 3

6. Chercher une chaîne dans un graphe

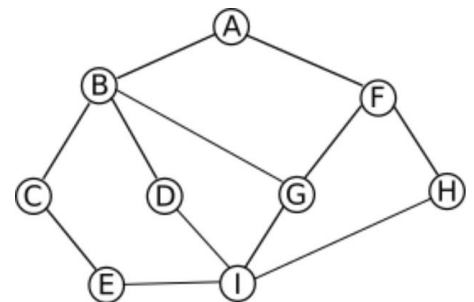
Nous allons maintenant nous intéresser à un algorithme qui permet de déterminer une chaîne entre deux sommets (sommet de départ et sommet d'arrivée). Les algorithmes de ce type ont une grande importance et sont très souvent utilisés (voir plus bas pour plus d'explications).

```
VARIABLES
G : un graphe
start : nœud (nœud de départ)
end : nœud (nœud d'arrivée)
u : nœud
chaîne : ensemble de nœuds (initialement vide)

DEBUT
TROUVE-CHAINE(G, start, end, chaîne):
    chaîne = chaîne U start // le symbole U signifie union, il permet d'ajouter le nœud start à
                           // l'ensemble chaîne
    si start est identique à end :
        renvoie chaîne
    fin si
    pour chaque sommet u adjacent au sommet start :
        si u n'appartient pas à chaîne :
            nchemin ← TROUVE-CHAINE(G, u, end, chaîne)
            si nchemin non vide :
                renvoie nchemin
        fin si
    fin pour
    renvoie NIL
FIN
```

1) Expliquer en quoi cet algorithme est un algorithme récursif.

2) Appliquer cet algorithme au graphe ci-contre, en prenant A pour sommet de départ et F pour sommet d'arrivée.



Il est également important de noter que, dans la plupart des cas, les algorithmes de recherche de chaîne (ou de chemin) travaillent sur des graphes pondérés (par exemple, pour rechercher la route entre un point de départ et un point d'arrivée dans un logiciel de cartographie). Ces algorithmes recherchent aussi souvent les chemins les plus courts (logiciels de cartographie). On peut citer l'algorithme de Dijkstra ou encore l'algorithme de Bellman-Ford (protocoles de routage) qui recherchent le chemin le plus court entre un nœud de départ et un nœud d'arrivée dans un graphe pondéré.

Parcours d'un graphe : travaux pratiques

Notebook « TP_Algorithmes_graphes.ipynb »

On commencera par implémenter, en Python, *graphe 1*, *graphe 2* et *graphe 3* à l'aide de dictionnaires.

Penser à proposer des jeux de tests pour chacune des fonctions implémentées.